



BSc (Hons) in **Information Technology**  
**Specialized in AI**

**IT3012 : Intelligent Agents**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## Practical 01

**Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

### Part 1: Code Analysis & PEAS Evaluation

#### Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

#### Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

The Four components in PEAS framework can be stated as Performance Measures, Environment, Actuators and Sensors.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.width, self.height, self.walls, self.food.positions, self.opponents, self.agent_pos
```

**3. (Analyze) Based on the variables initialized (specifically `self.opponents` ), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.**

The environment can be classified as a Multi-Agent Environment. The environment is identified as a Multi-Agent Environment because `self.opponents` initializes independent dynamic actors alongside the main player. These opponents exist as separate entities within the environment and can influence the outcome of the game through their actions. Since there is more than one agent operating in the environment, where the player must interact, compete, or coexist with other agents,

### **Task 1.2: The Perception Subsystem (`get_percept`)**

**4. (Remember) Define what a "percept sequence" is according to Lecture 01.**

Percept sequence is the complete history of everything the agent has perceived or sensed from its environment upto the very current moment. These historical records are strictly regarding the agent's sensory inputs or percepts over the time.

**5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?**

S-Sensors.

The method represents the Sensors(S) components of the PEAS Framework.

**6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.**

Partially Observable.

The environment is Partially Observable because the agent does not have complete information about the entire state of the environment through the `get_percept()` function.

### Task 1.3: Action Execution (execute\_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

P-Performance Measure

When `execute_action()` deducts points for hitting a wall, the Performance Measure (P) component of the PEAS model is being actively updated. This occurs because the agent's score is modified to reflect the quality of its actions.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important that the performance measure evaluates changes in the external environment state (such as hitting a wall) rather than the agent's internal processing effort because a performance measure is intended to assess what the agent accomplishes in its environment, not how much computation or effort it uses internally.

## Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

### Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Adding `self.toxic_traps` while hiding it from the agent's sensors keeps the environment Partially Observable. Since the agent cannot see the locations of the toxic traps through `get_percept()`, it does not have complete information about the environment. This increases uncertainty and forces the agent to make decisions with incomplete knowledge of the environment's state.

## Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding the `'smells_toxin'` sensor helps the agent act rationally by providing additional information about dangerous locations in the environment. This expands the agent's percept sequence and allows it to make better decisions. When the sensor detects a toxic tile, the agent can avoid actions that may lead to penalties and choose safer paths instead.

## Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

In Lecture 01, the classic "Vacuum World Exploit" illustrated the danger of a faulty performance measure that rewarded an agent for each piece of dirt cleaned rather than for maintaining a clean environment. As a result, a rational but reward-seeking agent could exploit the metric by repeatedly cleaning dirt and then creating or redistributing dirt to generate more cleaning opportunities and earn additional

Finalize and Lock Lab 01 Analytical Evaluation

**Lab 01 code analysis and theoretical evaluation complete and verified!**



**FACULTY OF COMPUTING**